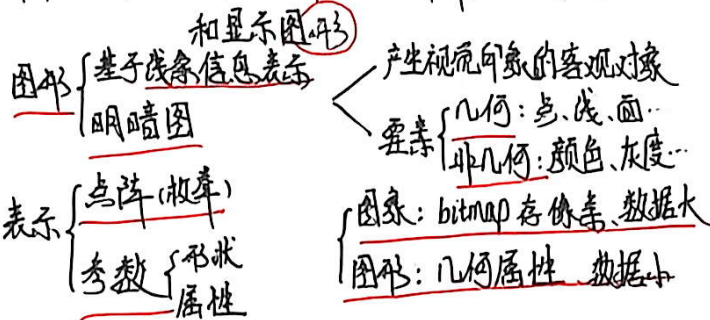


计算机图形学

一. 绪论

计算机图形学: 研究怎样用数字计算机生成、处理和显示图形



目标: 真实、实时、可控 → 即图像合成

图形学: 模型 → 图像 (逆过程)

计算机视觉: 图像 → 模型/信息分析 (逆过程)

图像处理: 对已有图像做增强、压缩、变换等

研究内容包括: 图形硬件、图形标准、图形交互、真实感渲染、动画等

二. 人眼系统与真实感

真实感渲染: 还原现实

- 几何真实性
- 光照与材质...
- 色彩与动态范围...
- 视觉舒适性...

颜色: 人眼对不同波长光能量的感知 (可见 380nm ~ 760nm)

谱分布: 光在各个可见波长的强度分布 (光谱同色)

由三刺激理论 → RGB 颜色空间 (三通道 (r, g, b))

$$C = rR + gG + bB \quad (r, g, b \text{ 浮点 } 8b \text{ 无符号})$$

RGB: 加色系统 (黑(0,0,0) 白(1,1,1)): 屏幕、电视 (主动发光)

CMY: 减色系统 (相反): 印刷 (被动发光)

CMYK: 多了 K 黑色通道

HSV: 更接近人眼颜色的直觉 (感知方式)

- H 色相
- S 饱和度
- V 亮度

图像: 二维离散函数 $f(x, y)$ 每个格子是一个像素

函数值是颜色

三角网格模型

- 顶点集 V
- 面片集 F (三角形)

顶点法向量: 周围面片法向量加权平均

显示设备 CRT: 电子偏转轰击发光

显示 $M \times N \times [log_2 K]$ 带宽 $M \times N \times F$

隔行扫描: 一帧分为两场

LCD: 体积小、功耗低

(可视角度、点距...)

空间频率: 图像在空间上变化多快

角频率: 每度视角内变化周期数 (距↑则↑) f_s/θ

时间频率: 图像随时间变化多快 (运动造成)

(速度矢量在空间梯度矢量上的投影) $v \cdot \nabla f$

人眼的频率响应: 视觉惰性 (高频闪看不出来)

临界闪烁频率: 约 46Hz

时空频率: 图像快速移动时, 人眼无法分辨空间细节

奈奎斯特定理: 采样频率 $f_s \geq 2f_{max}$ 则无损重建

采样后的信号频谱为原来的 $1/2$ 采样角频率 Ω_s 周期延拓

抗锯齿: 超采样、多重采样...

纹理过滤: 双/三线性...

客观真实: 科学准确性

主观真实: 看起来真

平衡

三. 计算机图形学基础

变换: 将空间中的一个点 x 映射成其它点 x' 的函数

图形学中点、边可用向量表示。矩阵可用来批量表达变换

矩阵乘的顺序重要

齐次坐标: (x, y) 写成 (x, y, h)

多一线来处理仿射变换的加法 → 全行乘

可表示无穷远点 $(x, y, z, 0)$

变换的分类: ① 刚体 (不变形) ② 相似 (缩放)

③ 线性 ④ 仿射 (平移) ⑤ 投影 (直线)

二维变换:

平移: $x' = x + T_x$ 缩放 (以 0 为中心): $x' = S_x x$

旋转 (θ 为逆旋角度): $x' = x \cos \theta - y \sin \theta$ $y' = x \sin \theta + y \cos \theta$ 错切: 使倾斜

⇒ 复合变换: 按顺序相乘 (把问题变为标准情形)

法向量不能总像点一样变: (切平面变换)

法向量变换矩阵 = 原变换矩阵的逆的转置

三维几何变换: 不同在于旋转要分绕哪个轴

(方向右手法则)

投影: 正交投影 $(x, y, z) \rightarrow (x, y, 0)$ (到 xy 平面)

透视投影 $(x, y, z) \rightarrow ((\frac{d}{z})x, (\frac{d}{z})y, d)$ (到 $z=d$)

用户域 > 窗口区 > 屏幕域 > 视图区

线性映射 (窗口边不平行则先转)

图形表示基础

三维形体模型 { 数据模型 (线框、表面、实体)
过程模型

分解表示: 把复杂形体分成不重叠规则子块
构造几何实体 CSG: 用基本体素, 通过交并补
边界表示 B-Rep: 记录点、边面和拓扑连接关系

曲线、样条和曲面:

曲线 $p(t) = (x(t), y(t))$

贝塞尔曲线由控制点定义 (通常不穿过所有)

三次有四个控制点

B样条是其推广 (局部可控)

拼接连续性: C0, G1, C1, C2

参数曲面 $p(u, v)$

双三次 Bézier 曲面片由 4×4 控制

旋转曲面: 曲线绕轴

广义圆柱: 曲线绕轴, 扫掠 (三维)

现代图形渲染管线: 把一堆顶点和几何数据, 经
变换、投影、光栅化、着色, 变为屏上彩色像素

四、直线、圆、椭圆生成算法

扫描转换 (光栅化): 确定一个像素集合, 用于显示一个图形

① 确定有关像素 ② 为颜色/其他属性

裁剪 → 扫描转换: 常用, 节省计算

扫描转换 → 裁剪: 算法简单

直线段:

DDA 算法: 利用“增量” $y(i+1) = y(i) + k$

从 (x_0, y_0) 开始, 每次 $\begin{cases} x = x+1 \\ y = y+k \end{cases}$ (取 $|k| \leq 1$)

实际点亮像素 $(x, \text{int}(y+0.5))$ (要舍入) (必为 float)

中点画线法: 设 $0 < k < 1$

已点亮 (x_p, y_p) , 则下一个可能是 $\begin{cases} (x_p+1, y_p) \\ (x_p+1, y_p+1) \end{cases}$

看二者中点: $(x_p+1, y_p+0.5)$ 在线上方/下方

记线为 $F(x, y) = ax + by + C = 0$, 则

$d = F(x_p+1, y_p+0.5)$ $\begin{cases} > 0, \text{线上, 取 } (x_p+1, y_p) \\ < 0, \text{线下, 取 } (x_p+1, y_p+1) \end{cases}$

d 加入增量更新 $\begin{cases} \text{取右 } d = d + a \\ \text{取右上 } d = d + a + b \end{cases}$ 初始 $d_0 = a + 0.5b$

避免小数: 都 $\times 2$

$\begin{cases} d_0 = 2a + b \\ d_1 = 2a \text{ (取右左)} \\ d_2 = 2(a+b) \text{ (取右右)} \end{cases}$

Bresenham 画线算法: 定义误差量 e 来判断直线在中点上方/下方
每走一步 x , e 增加 k ; 误差 $>$ 半个像素, 则 $y++$, $e-1$
(从 -0.5 开始, 若 > 0 则 $\uparrow$) ($\times 2dx$)

圆的扫描转换:

直接 $y = \pm \sqrt{R^2 - x^2}$ (开方、浮点、不均匀、取整)

角度 DDA $\begin{cases} x = x_0 + R \cos \theta \\ y = y_0 + R \sin \theta \end{cases}$

中点画圆法 (讨论第 2 个 1/8 圆)

$P = (x_p, y_p)$, 下一个 $\begin{cases} (x_p+1, y_p) \text{ 右} \\ (x_p+1, y_p-1) \text{ 右下} \end{cases}$ 取中点 M

$F(x, y) = x^2 + y^2 - R^2 \Rightarrow d = F(M)$ $\begin{cases} < 0, M \text{ 圆内, 取右} \\ > 0, M \text{ 圆外/上, 取右下} \end{cases}$

增量更新 $\begin{cases} \text{取右 } d = d + 2x_p + 3 \\ \text{取右下 } d = d + 2(x_p - y_p) + 5 \end{cases}$

初始点 $(0, R)$ $d_0 = 1.25 - R$ ($\times 4$ 变整)

Bresenham: 构造 $d_i = x_{hi}^2 + y_{hi}^2 + x_{li}^2 + y_{li}^2 - 2R^2$ $\begin{cases} < 0, \text{取右} \\ \geq 0, \text{取右下} \end{cases}$ (离弧最近的点)

增量 $\begin{cases} \text{取右 } d = d + 4x + 6 \\ \text{取右下 } d = d + 4(x - y) + 10 \end{cases}$ $d_0 = 3 - 2R$ $\begin{cases} < 0, \text{圆内, 外走 } x = x+1 \\ > 0, \text{圆外, 下走 } x = x, y = y-1 \end{cases}$

正负法画圆: 直观用 $F(x, y) = x^2 + y^2 - R^2$ $\begin{cases} < 0, \text{圆内, 外走 } x = x+1 \\ > 0, \text{圆外, 下走 } x = x, y = y-1 \end{cases}$

增量 $\begin{cases} F' = F + 2x + 1 \text{ 向右} \\ F' = F - 2y + 1 \text{ 向下} \end{cases}$

多边形逼近法: $\begin{cases} x_i = R \cos \theta_i \\ y_i = R \sin \theta_i \end{cases}$ 每次转角度 Δ (多边形逼近圆心的角)

等面积正多边形: 相交

椭圆: $F(x, y) = b^2x^2 + a^2y^2 - a^2b^2 = 0$ 一象限、分上下 (切-1) 个

$b^2(x_p+1) < a^2(y_p-0.5)$ 则在上, 否则在下部分 (中点)

上半: $F(M) < 0$, 取正右; > 0 , 取右下

下半: $F(M) < 0$, 取左下; > 0 , 取正下

五、光栅图形的扫描转换与区域填充

多边形的表示方法: 顶点表示、点阵表示

记录顶点序列 记录内部像素

矩形单独处理: 常用简单

扫描转换

共享边界归属: 左闭右开 下闭上开

(避免同一像素被反复填充或产生缝隙)

逐点判断法: 直观、最慢 → 遍历外接矩形的每一点

判断点是否在多边形内 $\begin{cases} \text{射线法: 任一方向射线, 交点数} \\ \text{累计角度法: 与各顶点连线} \\ \text{编码法} \end{cases}$

不可取 → 慢

扫描线算法 (利用相邻像素连续性)

处理非自交多边形

用一条水平线 $y=c$ 去切多边形

$\Rightarrow$ 产生若干交点, 从小到大排序

$\Rightarrow x_1, x_2, x_3, x_4, \dots$ 两两配对

$\Rightarrow$ 中间的区域为多边形内部

高效 { 区域的连续性 (内外交替)
扫描线的连续性

边的连续性 ($x_{new} = x_{old} + \frac{1}{k}$)

交点取整: 交点 x 为小数 { 左边界: 向右
右边界: 向左

边界像素取舍: 左闭右开, 右边界像素不填

扫描线穿过顶点: 奇点

分为 { 极值奇点: 两个 ($y_{i-1} - y_i, y_{in} - y_i$)
非极值奇点: 一个

引入两个数据结构, 高效实现扫描线算法:

边分类表 ET: 按边的下端点坐标 y 分类

有几条扫描线 对非水平边 同一类中按 x 递增排序
就设几类 每条边记录 { 上端点坐标 y_{max}
当前交点 x (初始下端点 x)
斜率倒数 $1/k$ (Δx)
next 指向下一条边

活化边表 AEL: 存当前扫描线正在相交的边
按交点 x 大小升序排

算法流程: 从 ET 中 最小的非空 y 开始 AEL 初为空

每到扫描线 y , 将 ET 中 y 中边加入 AEL

按 x 排序后两两配对填色

删去 $y = y_{max}$ 的边

$y = y+1, x = x + \Delta x$, 继续下一条

(快, 但要维护数据结构)

边缘填充算法: 利用性质 (取补 $\leftarrow$ 奇次变

偶次不变)

对于扫描线交点: 右方所有像素取补 (多 IO)

以扫描线为中心 以边为中心

栅格填充算法: 垂直扫描线 (一般过一顶点)

对交点和栅格之间的像素取补

减少重复访问 (不彻底)

边界标志算法: 先画边界再填内部

步骤: 对每条边作扫描转换, 打上边界标志

对每条扫描线从左往右访问

标志 $inside = false$, 遇边界取反, 为 true 填充

区域填充算法: 与边表示的区域 $\rightarrow$ 填色

要求区域连通 < 4 4 视为 8, 反之未必

内 4 边 8, 内 8 边 4 (防止斜角漏出去)

种子填充算法 (内点表示区域):

给种子 (x, y) 入栈, 出栈改色, 看上、下、左、右

在边界内且未改色则入栈 (栈的顺序! 先入后出!)

改进: 扫描线种子填充算法 (每段压栈一次)

种子入栈, 弹出点, 以其 y 为扫描线, 将区间 $[x_l, x_r]$

检查上、下扫描线在该区间是否有未填充

有则取每一个区间最右像素压入栈, 重复 (上 $\rightarrow$ 下, 左 $\rightarrow$ 右)

扫描转换与区域填充的比较:

基本思想 出发条件 对边界要求 也能相互转化

六、裁剪、反走样与消隐算法

裁剪: 确定图形哪些部分落在显示窗口内, 只显示这部分

点裁剪: 左下、右上两点, 判坐标

直线段裁剪: 在内里、在外舍、内外看交点

Cohen-Sutherland 裁剪

首尾 code $<$ 全 0, 在内

相与不为 0, 在外

中点分割裁剪: code 直判, 用二分逼近交点

Liang-Barsky: 与边界 (延长线) 求交, 找距线段首尾最近点

计算交点参数

Cyrus-Beck (凸多边形窗口): 对每边, 取一点 A 及法向 N

多边形裁剪: 可能不封闭, 需窗口边界一部分补上

Sutherland-Hodgman 算法: 一次只用一条边裁 (左上右下)

设右可见

输出 P 不输出 I, P 输出 I

对凹: 分为凸、修算法、WA

反走样: 主样指用离散像素表示连续图形时的失真 (采样不足)

减少/消除

提高分辨率 (代价大)

简单区域采样: 根据相交的面积确定亮度

加权区域采样: 像素内部权重不一样

消隐: 消除被遮挡的不可见线或面

(z 值 $\rightarrow$ z 值, 丢失深度信息)

(处理 z 值 依赖视点)

按对象 { 面消隐
线消隐

面消隐 (按处理空间) { 图像空间 (像素) $O(mnhk)$
物体空间 (物体) $O(ckh)^2$

z -buffer 算法: { 帧缓冲器: 各像素颜色
 z 缓冲器: 各像素处物体深度值

帧 b 初始化为背景色, z_b 初始化为最小

遍历每个多边形, 对覆盖像素 (x, y) 算深度

若 $z(x, y)$ 大于 z_b 值, 说明更近, 更新两 b

简单稳定、计算量大

改进: 不存整个 z_b , 对每 (x, y) 看所有多边形

扫描线 z -buffer: 对每条扫描线处理

(z 值 z_b) 多边形分类表 PT 活化多边形表 APL
深度可增量

区间扫描线算法: 扫描线被分成多个区间

区间被 $0 \sim$ 多个多边形覆盖

深度最大的填充

不能相互贯穿

光线投射算法: 对每个像素, 从视点发射线
由交点确定 (模拟视觉)

算法权衡: 计算速度与图形细节

七、真实感渲染技术

真实感图形要表现质感、层次、细微变化等因素

光照模型: 计算物体表面某一点光强/颜色

局部
全局

简单光照模型:

环境光 $I_e = K_a I_a$ (物体各点一样)

漫反射 $I_d = I_p K_d \cos \theta$ $\cos \theta = L \cdot N$

镜面反射和高光 $I_s = I_p K_s \cos^2 \alpha$ $\max(0, \sim)$



n 控制高光大小、光滑程度

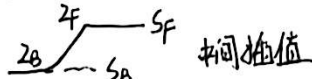
Phong 光照模型: z 者之和

近似: 用 L, V 的平均 H , $H \cdot N$ 近似 $R \cdot V$

光的衰减和深度暗示

(值随衰减函数 $f(d)$)

深度暗示: 从人眼到物体



彩色场景和多光源

对 a, b 分别 贡献漫反射和镜面反射, 求和

Phong 不足: 经验模型、像塑料、未考虑间接反射光、不够真

网格着色: 小 z 三角形网格近似 (整个一样: 平坦着色)

Gouraud 着色: 快 可能丢高光 依赖划分

每个多边形单位法向量 $\rightarrow$ 用相邻的平均, 得顶点单位法向量

$\rightarrow$ 每个顶点用光照模型算 $\rightarrow$ 内部线性插值 (双线性)

Phong 着色: 法向插值 慢 更真实

得到多边形内部每个点的法向量, 逐点算颜色

透明效果:

插值透明: $I = (1 - k_{t1}) I_1 + k_{t1} I_2$ (只对环境光和漫反射)

过滤透明: 滤光器 C_{tx} 表示对某种颜色

$I_{\lambda} = I_{\lambda 1} + k_{t1} C_{tx} I_{\lambda 2}$ (k_{t1} 也不再仅 $(0, 1)$)

考虑折射的透明: $\frac{\sin \theta_1}{\sin \theta_2} = \frac{n_1}{n_2}$

全局光照和 Whitted 模型 (考虑间接贡献)

$I_{\lambda} = I_{\lambda 1} + K_s I_{s \lambda} + K_t I_{t \lambda}$

局部 反射 折射

光线跟踪: 视点 $\rightarrow$ 每个像素, 找最近交点 P

P 的局部 M 发射/透射 (递归计算)

阴影: 空间表示

阴影

接收者 遮挡物 本影 半影

附着阴影 (背离)
投射阴影 (被挡)
自阴影 (同一)

硬阴影: z 值 (在/不在)

软阴影: 来自有面积光源 边缘柔和

平面阴影: 把遮挡物体表面的点沿光源方向投影到平面上

构造 M 使 $MV = P$

阴影体: 点光源经轮廓延伸 $\rightarrow$ 阴影体

从视点发射线到某点, 经体 $+1$ 出 -1 , >0 则在阴影中

一个模板缓存 效率低、精度高 (边界清晰)

阴影图: 基于 z_b

第一遍: 光源当相机, 记录各方向物体深度

二: 正常渲染, 可见点变换到光源视角, 距离大于

深度则说明在阴影中

速度快、可能锯齿、失真、硬件支持好

八. AI加速的实时渲染

深度学习超采样

TAA: 多帧抖动采样 + 运动矢量重投影 + 历史累积来抗锯齿
(模糊 鬼影 闪烁...)

时间超分辨率: 低分辨率渲染, 多帧累积重插

神经超采样: 特征输入, 神经网络重插

NSRR

深度学习帧生成: 算法补帧

视频插帧 VFI

内插 / 外推

深度学习特卡洛去噪: 低采样有噪声

传统滤波 / 神经网络

九. 传统交互技术

自然、实时、可控的操作

交互需求

意图 → 执行 → 处理、反馈 → 感知、评价

↑
输入

↑
输出

VR 虚拟现实 (沉浸、交互、想象)

AR 增强现实 (虚+实) (虚实融合、实时交互、三维配准)

十. 真实图形学应用

医疗健康: 客观真实 CT, MIR

2D → 3D

CAD与工业设计: 主观真实 + 精确建模

参数化、特征 CSG B-Rep

影视制作: 主观真实